

Compiladores

Practica 1 : Regex a NFA

- * Pérez Carranza Yosdi Jeremy 319296594
- * Godinez Hernandez Axel Alejandro.....321072489
- * Calva García José Luis 321166584

10 de septiembre de 2026

Validación

Axel Alejandro Godinez Hernandez

Validador _ Practica 1 Godinez Axel.pdf

José Luis Calva García

Calva García José Luis

Thompson Shunting yard

Recorridos

1) a.b.c.d.e.f.g

2) a.b.c.e.t

3) a.b.c.t.d.e

* g

4) (a.b.c.d.e.f.g.t)*b

5) (a.b.c.d.e.f.g.t)*b

6) (a.b.c.d.e.f.g.t)*b

7) (a.b.c.d.e.f.g.t)*b

8) (a.b.c.d.e.f.g.t)*b

9) (a.b.c.d.e.f.g.t)*b

10) (a.b.c.d.e.f.g.t)*b

11) (a.b.c.d.e.f.g.t)*b

12) (a.b.c.d.e.f.g.t)*b

13) (a.b.c.d.e.f.g.t)*b

14) (a.b.c.d.e.f.g.t)*b

15) (a.b.c.d.e.f.g.t)*b

16) (a.b.c.d.e.f.g.t)*b

17) (a.b.c.d.e.f.g.t)*b

18) (a.b.c.d.e.f.g.t)*b

19) (a.b.c.d.e.f.g.t)*b

20) (a.b.c.d.e.f.g.t)*b

21) (a.b.c.d.e.f.g.t)*b

22) (a.b.c.d.e.f.g.t)*b

23) (a.b.c.d.e.f.g.t)*b

24) (a.b.c.d.e.f.g.t)*b

25) (a.b.c.d.e.f.g.t)*b

26) (a.b.c.d.e.f.g.t)*b

27) (a.b.c.d.e.f.g.t)*b

28) (a.b.c.d.e.f.g.t)*b

29) (a.b.c.d.e.f.g.t)*b

30) (a.b.c.d.e.f.g.t)*b

31) (a.b.c.d.e.f.g.t)*b

32) (a.b.c.d.e.f.g.t)*b

33) (a.b.c.d.e.f.g.t)*b

34) (a.b.c.d.e.f.g.t)*b

35) (a.b.c.d.e.f.g.t)*b

36) (a.b.c.d.e.f.g.t)*b

37) (a.b.c.d.e.f.g.t)*b

38) (a.b.c.d.e.f.g.t)*b

39) (a.b.c.d.e.f.g.t)*b

40) (a.b.c.d.e.f.g.t)*b

41) (a.b.c.d.e.f.g.t)*b

42) (a.b.c.d.e.f.g.t)*b

43) (a.b.c.d.e.f.g.t)*b

44) (a.b.c.d.e.f.g.t)*b

45) (a.b.c.d.e.f.g.t)*b

46) (a.b.c.d.e.f.g.t)*b

47) (a.b.c.d.e.f.g.t)*b

48) (a.b.c.d.e.f.g.t)*b

49) (a.b.c.d.e.f.g.t)*b

50) (a.b.c.d.e.f.g.t)*b

51) (a.b.c.d.e.f.g.t)*b

52) (a.b.c.d.e.f.g.t)*b

53) (a.b.c.d.e.f.g.t)*b

54) (a.b.c.d.e.f.g.t)*b

55) (a.b.c.d.e.f.g.t)*b

56) (a.b.c.d.e.f.g.t)*b

57) (a.b.c.d.e.f.g.t)*b

58) (a.b.c.d.e.f.g.t)*b

59) (a.b.c.d.e.f.g.t)*b

60) (a.b.c.d.e.f.g.t)*b

61) (a.b.c.d.e.f.g.t)*b

62) (a.b.c.d.e.f.g.t)*b

63) (a.b.c.d.e.f.g.t)*b

64) (a.b.c.d.e.f.g.t)*b

65) (a.b.c.d.e.f.g.t)*b

66) (a.b.c.d.e.f.g.t)*b

67) (a.b.c.d.e.f.g.t)*b

68) (a.b.c.d.e.f.g.t)*b

69) (a.b.c.d.e.f.g.t)*b

70) (a.b.c.d.e.f.g.t)*b

71) (a.b.c.d.e.f.g.t)*b

72) (a.b.c.d.e.f.g.t)*b

73) (a.b.c.d.e.f.g.t)*b

74) (a.b.c.d.e.f.g.t)*b

75) (a.b.c.d.e.f.g.t)*b

76) (a.b.c.d.e.f.g.t)*b

77) (a.b.c.d.e.f.g.t)*b

78) (a.b.c.d.e.f.g.t)*b

79) (a.b.c.d.e.f.g.t)*b

80) (a.b.c.d.e.f.g.t)*b

81) (a.b.c.d.e.f.g.t)*b

82) (a.b.c.d.e.f.g.t)*b

83) (a.b.c.d.e.f.g.t)*b

84) (a.b.c.d.e.f.g.t)*b

85) (a.b.c.d.e.f.g.t)*b

86) (a.b.c.d.e.f.g.t)*b

87) (a.b.c.d.e.f.g.t)*b

88) (a.b.c.d.e.f.g.t)*b

89) (a.b.c.d.e.f.g.t)*b

90) (a.b.c.d.e.f.g.t)*b

91) (a.b.c.d.e.f.g.t)*b

92) (a.b.c.d.e.f.g.t)*b

93) (a.b.c.d.e.f.g.t)*b

94) (a.b.c.d.e.f.g.t)*b

95) (a.b.c.d.e.f.g.t)*b

96) (a.b.c.d.e.f.g.t)*b

97) (a.b.c.d.e.f.g.t)*b

98) (a.b.c.d.e.f.g.t)*b

99) (a.b.c.d.e.f.g.t)*b

100) (a.b.c.d.e.f.g.t)*b

Pérez Carranza Yosdi Jeremy 10-09-2026 Scríbe

Ejercicio de Validación. Algoritmo de Shunting-Yard.

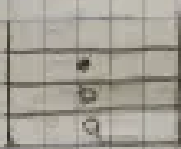
$(a b c d)^+ \rightarrow (a \cdot b \cdot c \cdot d)^+$

1. Op: (	5. Op: (.	9. Op: +
Out: a	Out: ab.c	Out: ab.c.d.
2. Op: (.	6. Op: (.	10. Op:
Out: ab	Out: ab.c.	Out: ab.c.d.+
3. Op: (.	7. Op: (.)	
Out: ab	Out: ab.c.d	
4. Op: (.	8. Op:	
Out: ab.	Out: ab.c.d.	

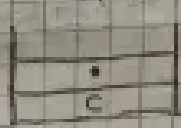
Out: ab.c.d.+ Algoritmo de Thompson.

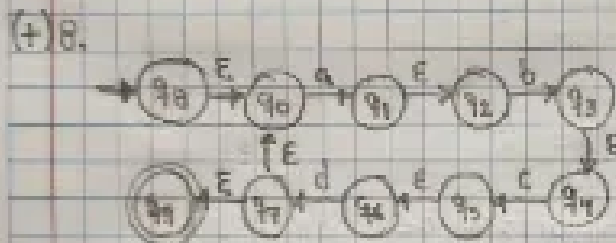
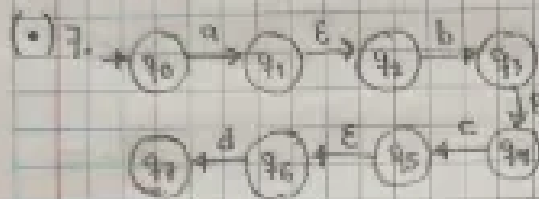
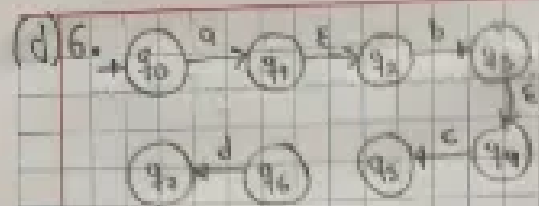
(a) 1. Pila: $\rightarrow q_0 \xrightarrow{a} q_1$ 

(b) 2. Pila: $\rightarrow q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$ 

(c) 3. $\rightarrow q_0 \xrightarrow{a} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{b} q_3$ Pila: 

(c) 4. $\rightarrow q_0 \xrightarrow{a} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{b} q_3 \xrightarrow{c} q_4 \xrightarrow{\epsilon} q_5$ Pila: 

(c) 5. $\rightarrow q_0 \xrightarrow{a} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{b} q_3 \xrightarrow{c} q_4 \xrightarrow{\epsilon} q_5$ Pila: 



¿Tenemos que la cadena $abcdabed$ es aceptada?

Inicio: $q_3 \xrightarrow{\epsilon} q_0$ $\{q_0, q_3\}$

1. 'a' $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{\epsilon} q_2$ $\{q_1, q_2\}$
2. 'b' $q_2 \xrightarrow{b} q_3$, $q_3 \xrightarrow{\epsilon} q_4$ $\{q_3, q_4\}$
3. 'c' $q_4 \xrightarrow{c} q_5$, $q_5 \xrightarrow{\epsilon} q_6$ $\{q_5, q_6\}$
4. 'd' $q_6 \xrightarrow{d} q_7$, $q_7 \xrightarrow{\epsilon} q_0$, $q_0 \xrightarrow{\epsilon} q_3$ $\{q_7, q_0, q_3\}$
5. 'a' $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{\epsilon} q_2$ $\{q_1, q_2\}$
6. 'b' $q_2 \xrightarrow{b} q_3$, $q_3 \xrightarrow{\epsilon} q_4$ $\{q_3, q_4\}$
7. 'c' $q_4 \xrightarrow{c} q_5$, $q_5 \xrightarrow{\epsilon} q_6$ $\{q_5, q_6\}$
8. 'd' $q_6 \xrightarrow{d} q_7$, $q_7 \xrightarrow{\epsilon} q_0$, $q_0 \xrightarrow{\epsilon} q_3$ $\{q_7, q_0, q_3\}$

∴ El ciclo final de estados activos es $\{q_7, q_0, q_3\}$ y dado que el estado final de aceptación q_8 se encuentra dentro del ciclo final, el NFA acepta la cadena y pertenece al lenguaje.

Descripción de implementación

Definimos primeramente la estructura de State (estado) como una estructura recursiva para poder ligar los estados de nuestro autómata. Contiene un carácter con el que cambia a otro estado, dos punteros a estados (salida1 y salida2) debido a que por la estrella de Kleene y otras operaciones hay estados que bifurcan a dos estados distintos, y un bool de visitado para poder liberar la memoria del NFA y evitar ciclos.

Posteriormente, consideramos la estructura de nfa como un conjunto delimitado por dos punteros: estado_inicial y estado_final, ya que nos importa ir cambiando y conectando los estados finales e iniciales de los sub-autómatas.

Para la parte de las expresiones regulares, definimos un Item como un elemento de una expresión regular que tiene un char como valor. Una expresión regular (regex) es entonces un arreglo dinámico de ítems con un tamaño dado. Esta estructura en la parte de regex nos permitió definir una cadena con un tamaño definido de ayuda para poder realizar los siguientes algoritmos.

Algoritmo de Shunting-Yard.

Este es el primer algoritmo y proceso por el cuál partiremos, antes de transformar una expresión regular en un autómata finito no determinista (NFA), primero fue necesario convertir la entrada original (que viene en notación infija tradicional) a notación posfija. Esto facilita el procesamiento secuencial con el algoritmo de Thompson sin tener que lidiar con paréntesis y devolver la expresión adaptada para ser procesada en el algoritmo de Thompson. En los archivos regex.h y regex.c se utilizaron estructuras de datos eficientes:

- ☐ Item: Un struct básico que contiene un solo carácter (char value). Sirve como el bloque elemental para representar los símbolos de la expresión regular.
- ☐ regex: Un struct compuesto por un puntero a un arreglo dinámico de Item (Item *items) y un entero (int size) que indica la cantidad de elementos almacenados. Es la estructura final que se devuelve para continuar con el autómata.
- ☐ Pila auxiliar (Stack): Dentro de la función parse_regex implementamos una pila temporal mediante un arreglo de caracteres (char *pila) y una variable entera (top_pila) como índice de posición. Esta pila se usa para guardar los operadores y los paréntesis mientras determinamos su orden correcto.

El procesamiento de la expresión se dividió en dos pasos consecutivos dentro del archivo `regex.c`: Implementamos la función `insertar_concatenacion_explicita` que recorre la cadena e inserta un punto `.` entre dos caracteres cuando detecta que termina un bloque y comienza otro bloque (como una letra o un `"(`). Con la cadena ya formateada con puntos, aplicamos el algoritmo usando una función de precedencia para asignar valores numéricos a los operadores (`*,+,?= 3, . = 2, | = 1`) y luego tenemos que:

1. Si leemos un símbolo (letra/número), pasa directo a la cadena de salida.
2. Si leemos un `(`, entra directo a la pila.
3. Si leemos un `)`, sacamos elementos de la pila hacia la salida hasta encontrar el `(`.
4. Si leemos un operador (`*, ., |`), comparamos su prioridad con el que está en el tope de la pila. Mientras el tope tenga mayor o igual prioridad, lo sacamos hacia la salida. Después, guardamos el nuevo operador en la pila.
5. Al terminar la lectura, vaciamos los operadores restantes de la pila a la salida.

Decidimos separar el formateo de la cadena (inserción del `.` en la expresión) de la lógica de Shunting-Yard en funciones distintas. Esto hizo que el código fuera mucho más fácil de comprender y también, asignamos memoria de forma dinámica con `malloc` para manejar cadenas de tamaño variable, y nos aseguramos de liberar con `free` los arreglos temporales (`exp_format`, `salida`, `pila`) antes de retornar la estructura `regex`, evitando fugas de memoria.

Algoritmo de Thompson

En el algoritmo de Thompson realizamos una construcción paso a paso de un autómata a partir de una expresión regular (`regex`) posfija en la que nos apoyamos de una pila. Para ello, nos ayudamos de una función auxiliar (`crear_estado`) que asigna las conexiones y símbolos de transición a un nuevo estado. El algoritmo inicia cuando vamos leyendo los caracteres de la expresión `regex` (posfija) definida anteriormente y para cada ítem obtenemos su `char` de ahí con base en el símbolo dado realizamos las siguientes acciones:

- ☐ Símbolos Básicos (a-z, A-Z, 0-9): Si es un valor alfanumérico, creamos el autómata correspondiente con un estado inicial que tiene una transición a un estado final mediante el `char` dado y lo metemos a la pila.
- ☐ Concatenación(`.`): Obtenemos de la pila los dos NFA a unir (`N2` y luego `N1`). Debido a que recibimos una expresión posfija, sabemos que si llega una operación los valores estarán a la izquierda. Modificamos el estado final de `N1` para que su transición sea ϵ hacia el estado inicial de `N2`, creando un nuevo NFA que va desde el inicio de `N1` al final de `N2`, y lo metemos a la pila.

- ☐ Unión / OR ($\cup$): Obtenemos los dos NFA de la pila (N2 y N1). Creamos un nuevo estado inicial que vaya con una transición ϵ a N1 y otra a N2, y un estado final nuevo al que convergen los finales de ambos autómatas con transiciones ϵ . Metemos el nuevo NFA a la pila.
- ☐ Estrella de Kleene (*): Obtenemos un NFA de la pila. Creamos un nuevo estado inicial y un nuevo estado final. El nuevo inicio va por ϵ al inicio del NFA y también al nuevo final (para permitir 0 repeticiones). El estado final del NFA original se modifica para que regrese por ϵ a su inicio (para repetir) y al nuevo final. Metemos el NFA resultante a la pila.
- ☐ Cerradura positiva (+): Obtenemos un NFA de la pila. Creamos un nuevo estado inicial que va por ϵ directamente al inicio del NFA (para obligar a que pase al menos 1 vez) y un nuevo estado final. El final del NFA original puede regresar por ϵ a su inicio o ir al nuevo final. Metemos el NFA a la pila.
- ☐ Opcional (?): Obtenemos un NFA de la pila. Creamos un nuevo estado inicial que va por ϵ al inicio del NFA (1 ocurrencia) o al nuevo estado final (0 ocurrencias). El estado final del NFA original se conecta mediante ϵ al nuevo final. Metemos el NFA resultante a la pila.

Algoritmo match_nfa.

Para poder implementar nuestra función match_nfa encargada de devolver un 1 en caso de existir o 0 en caso no existir una coincidencia entre una cadena y un autómata finito no determinista. Primero se implementó una función agregar_estado que recibiendo un estado, una lista de estados, una lista de estados marcados, como números que funcionaran como puntero tanto para nuestra lista de estados y estados marcados, donde básicamente iremos agregando un estado a una posible ruta siempre que cumpla los requisitos de que no sea un estado nulo o visitado y posteriormente si sus salidas al tener transición epsilon se agrega para poder llegar a un cierre epsilon en nuestra ruta.

Por lo que seguido, empezamos nuestra solución para match_nfa primero definiendo listas de estados “actuales”, “siguientes” y “marcados” con sus respectivos punteros enteros cada uno, primero empezando tomando nuestro estado inicial y calculamos que otros estados podemos llegar por medio de agregar_estado, guardando en nuestra lista de “actuales” y “marcados” limpiando su marca de visitados para poder usarlos posteriormente.

Luego por medio de un bucle for iremos revisando nuestra cadena de texto de caracter por caracter. Donde por cada estado que estemos parados, revisaremos si tiene una transición que coincidan con nuestra letra actual, para agregar este estado destino a nuestra lista “actuales”. Seguido respetaremos las banderas de visitado en nuestros estados visitados con el fin de no interferir en nuevas posibles búsquedas. Y por cada turno sera necesario pasar cada estado en la lista “siguiente” a la lista “actuales” para nuestra siguiente ronda en búsqueda de una ruta que



satisfaga nuestra conciencia de autómeta y cadena. Y finalmente en caso de llegar el caso de quedarnos sin ningun estado en la lista “actuales”, podemos retornar un valor ‘0’ como respuesta de que no existe coincidencia. En caso contrario, si revisamos dentro de nuestra lista de “actuales” y logramos una ruta que logre alcanzar nuestro estado final devolvemos un valor ‘1’ como respuesta de una coincidencia de autómeta y cadena.

Para poder implementar tambien `save_nfa` necesitamos una funcion auxiliar `numerar_estados` que por medio de un estado, una lista de estado y un número puntero se encargara de ir guardando en nuestra lista todo estado no visitado o no nulo, como dandole un número único de identificación a cada estado.

Para nuestra implementación concreta de `save_nfa`, realizamos las validaciones básicas de seguridad comprobando que la estructura del autómeta exista y abre el archivo de destino en modo escritura. A continuación, utiliza la función auxiliar `numerar_estados` para recorrer todo el autómeta a partir del estado inicial; esto permite consolidar todos los estados en un arreglo (lista) y asignarles a cada uno un número entero único (un índice del 0 al total-1). Luego, busca en esa lista en qué posición quedó el estado final para conocer su índice asignado.

Posteriormente, escribimos nuestro encabezado en el archivo donde se especifica cual es el estado inicial, el inicio de su estado final y la cantidad total de estados registrados. Siendo nuestro contenido central, recorre cada uno de los estados numerados e inspeccionando sus punteros de salida. Incluso imprimiendo la transacción, de cada estado especificando si es por medio de un carácter o un salto epsilon.

Por último, recorre nuevamente la lista para restablecer la variable visitada a false en cada estado, garantizando que el autómeta no quede marcado y se pueda volver a recorrer en operaciones futuras. Finalmente, cierra el archivo correctamente y retorna true para confirmar que el proceso de guardado fue exitoso.

Dificultades Encontradas

Para comenzar, la primera dificultad que tuvimos fué identificar las reglas de concatenación, saber exactamente cuándo debía colocarse un punto “.” y cuándo no. Al principio intentamos usar muchos condicionales `if` anidados que volvían el código confuso. Para solucionar esto, simplificamos la lógica dividiendo las condiciones en dos variables booleanas (`c1_es_final` y `c2_es_inicio`). Si el carácter actual marcaba el final de un elemento y el siguiente marcaba el inicio de otro, sabíamos con certeza que se debía insertar un punto “.” ; También algunos operadores binarios salían en orden invertido cuando tenían la misma prioridad, por lo que ajustamos la condición del ciclo `while` en Shunting-Yard para usar `>=` en lugar de `>`, asegurando que si dos operadores tienen la misma precedencia (como dos puntos de concatenación seguidos), el que ya estaba guardado en la pila salga primero hacia la salida.

Otra de las primeras dificultades que enfrentamos fue decidir qué tipo de estructura utilizar para representar los estados del autómata, ya que requerían características especiales. La forma de ligarlos entre sí fue un factor decisivo: aunque inicialmente consideramos usar una lista simple, necesitábamos una estructura capaz de manejar bifurcaciones donde un mismo estado pudiera apuntar a dos estados distintos.

Vinculado a esta estructura, otro reto importante fue diseñar la estrategia para vaciar el NFA de la memoria. Teníamos que idear un algoritmo de liberación que no se ciclara indefinidamente al encontrarse con referencias cruzadas o bifurcaciones. En un principio pensamos en almacenar los estados en una lista auxiliar para gestionarlos, pero finalmente optamos por agregar un campo booleano `visitado` a cada estado (inicializado en `false`). Esto nos permitió liberar la memoria de forma mucho más limpia y segura, recorriendo y liberando únicamente los nodos que no habían sido marcados.

Por otro lado, plasmar la idea teórica de los algoritmos directamente en el lenguaje C representó un obstáculo, ya que la sintaxis y el manejo riguroso de punteros nos resultaba algo engorroso. Finalmente, la integración con Docker fue un obstáculo importante para el equipo, al ser una herramienta nueva para varios miembros o haberla utilizado previamente de formas distintas nos generó ciertos contratiempos durante el desarrollo de la práctica.

Resultados Obtenidos

Incluimos la captura de pantalla del validador ejecutado mediante Docker:

```
Last login: Wed Sep  9 18:32:52 on ttys002
axelgodinez@MacBook-Air-de-Axel-2 regex_to_nfa % docker run --rm --platform linux/amd64 regex_to_nfa_validator
-- The C compiler identification is GNU 15.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done (1.1s)
-- Generating done (0.0s)
-- Build files have been written to: /app
[ 25%] Building C object CMakeFiles/regex_to_nfa.dir/src/main.c.o
[ 50%] Building C object CMakeFiles/regex_to_nfa.dir/src/nfa.c.o
[ 75%] Building C object CMakeFiles/regex_to_nfa.dir/src/regex.c.o
[100%] Linking C executable regex_to_nfa
[100%] Built target regex_to_nfa
Regex: a|ba*b(a|b)*
  postfix ok
  nfa ok: 11100
Regex: a(b|c)*d
  postfix ok
  nfa ok: 1110
Regex: (a|b)*c
  postfix ok
  nfa ok: 110
Regex: ab
  postfix ok
  nfa ok: 100
Regex: a|b|c
  postfix ok
  nfa ok: 1110
Regex: a*
  postfix ok
  nfa ok: 1110
Regex: ab+c
  postfix ok
  nfa ok: 110
Regex: a?b
  postfix ok
  nfa ok: 110
Regex: (ab)*
  postfix ok
  nfa ok: 1110
Regex: (a|b)c+d
  postfix ok
  nfa ok: 1110

Overall: PASS
axelgodinez@MacBook-Air-de-Axel-2 regex_to_nfa %
```